

Hashing

1. Why Hashing?

Motivation

Direct Addressing wastes memory when key range is very large. Hashing reduces memory while keeping operations nearly constant time.

Instead of storing elements at index k , we compute a smaller index using a **hash function**.

$$h(k) = k \mod m$$

Where:

- k = key
- m = size of hash table

2. Example of Hash Function

Let:

$$m = 10$$

Keys:

10, 22, 31, 4, 15

Compute hash values:

$$h(10) = 0$$

$$h(22) = 2$$

$$h(31) = 1$$

$$h(4) = 4$$

$$h(15) = 5$$

Visual Representation

0	1	2	3	4	5	6	7	8	9
10	31	22		4	15				

Memory usage is now:

$$O(m)$$

instead of $O(U)$.

3. Collision

Definition of Collision

A collision occurs when two different keys map to the same index.

Mathematically:

$$h(k_1) = h(k_2), \quad k_1 \neq k_2$$

Collision Example

Let:

$$m = 10$$

Keys:

$$22, 32$$

$$h(22) = 2$$

$$h(32) = 2$$

Both map to index 2 $\rightarrow$ Collision.

0	1	2	3	4	5	6	7	8	9
		22 32							

4. When Is Collision Guaranteed?

If:

$$n > m$$

Where:

- n = number of keys
- m = table size

Then collision must occur.

Pigeonhole Principle

If more objects than boxes, at least one box contains more than one object.

Therefore:

Collisions are unavoidable in hashing.

5. Properties of a Good Hash Function

Goal

A good hash function distributes keys uniformly across the table to minimize collisions.

1. Uniform Distribution

A good hash function should spread keys evenly over:

$$0, 1, 2, \dots, m - 1$$

If keys are random, then each index should be equally likely:

$$P(h(k) = i) = \frac{1}{m}$$

Why important?

- Reduces clustering
- Reduces collisions
- Ensures average $O(1)$ performance

2. Deterministic

For the same key k :

$h(k)$ must always produce the same value

Otherwise searching becomes impossible.

3. Fast to Compute

Hash function must be:

$$O(1)$$

It should use simple arithmetic operations.

4. Uses All Key Information

A good hash function should depend on all bits of the key.

Small change in key $\rightarrow$ large change in hash value.

This property is called:

Avalanche Effect

Example:

$$h(12345) \neq h(12346)$$

5. Minimizes Patterns

If input keys follow a pattern, hash output should not preserve that pattern.

Example:

Sequential keys should not map to sequential indices.

6. Examples of Poor Hash Functions

Poor Hash Functions Cause Heavy Collisions

Bad hash functions destroy performance.

Example 1: Constant Hash Function

$$h(k) = 1$$

All keys map to index 1.

Result:

- Maximum collisions
- Time complexity becomes $O(n)$
- Equivalent to linked list

Worst possible hash function.

Example 2: Using Only Last Digit

$$h(k) = k \bmod 10$$

If table size $m = 10$ and keys are:

10, 20, 30, 40, 50

Then:

$$h(10) = 0$$

$$h(20) = 0$$

$$h(30) = 0$$

All collide at index 0.

Poor distribution.

Example 3: Ignoring Higher Bits

Suppose keys are large numbers but we use:

$$h(k) = k \bmod 8$$

If $m = 8$ and keys differ only in higher bits:

256, 512, 1024

All are multiples of 256:

$$h(256) = 0$$

$$h(512) = 0$$

$$h(1024) = 0$$

Severe clustering.

Example 4: Sequential Mapping

$$h(k) = k$$

If table size is small and keys are large sequential numbers, collisions occur frequently.

Better:

$$h(k) = k \bmod m$$

Comparison: Good vs Poor Hash Function

Property	Good	Poor
Uniform distribution	✓	✗
Uses full key	✓	✗
Fast computation	✓	✓
Avoids clustering	✓	✗
Maintains $O(1)$ avg	✓	✗

A Hash Table is Only as Good as its Hash Function

Hashing = Memory Efficient + Near Constant Time